



CURSO: TSI (Tecnologia em Sistemas para Internet)

Lista de Exercícios – Funções

Data: 24/10/2018

Disciplina: TSI31B-Fundamentos de Programação

Semestre: 1

Professor: Fabio Alexandre SPANHOL, fspanhol@gmail.com

Turma: SI13A

Período: 1

Turno: Noturno

Nome:

RA:

***Questões discursivas serão avaliadas considerando coerência com o assunto solicitado, clareza, completude e embasamento teórico- técnico.

*Essa lista pode ser resolvida em trio.

1. Implemente a função `fDecToBin (n)`, que devolve uma string contendo a representação binária do decimal inteiro `n`. Por exemplo, `fDecToBin (6)` deve retornar "110".
2. Escreva uma função chamada `fCountVowel` que recebe uma string e retorna a quantidade de vogais existentes na string.
3. Implemente uma função que recebe um número inteiro positivo qualquer e retorna a quantidade de algarismos que o mesmo possui.
4. Escreva uma função `fSomQuad` que recebe um inteiro `n` qualquer e retorna a soma dos quadrados de todos os inteiros positivos menores que `n`.
5. A conversão de graus Fahrenheit para centígrados é obtida pela fórmula $C = 5/9 * (F - 32)$. Implemente uma função `fFahrenheit2Celsius (c)` que recebe um valor em graus centígrados e devolve-o convertido para graus Fahrenheit. Usando essa função, escreva um programa que calcule e escreva uma tabela de graus centígrados, em função de graus Fahrenheit que variem de 50 a 150, de 1 em 1.
6. Implemente a função `fDecToOct (n)`, que devolve uma string contendo a representação octal do decimal inteiro `n`. Por exemplo, `fDecToOct (10)` deve retornar "12".
7. Implemente a função `char * fDecToHex (n)`, que devolve uma string contendo a representação hexadecimal do decimal inteiro `n`. Por exemplo, `fDecToHex (254)` deve retornar "FE".
8. Implemente a função `fHexToDec (s)`, que devolve um inteiro decimal correspondente a representação hexadecimal passada no parâmetro `s`. Por exemplo, `fHexToDec ("FE")` deve retornar 254.
9. Implemente a função `fOctToDec (s)`, que devolve um inteiro decimal correspondente a representação octal passada no parâmetro `s`. Por exemplo, `fOctToDec ("12")` deve retornar 10.
10. Implemente a função `fBinToDec (s)`, que devolve um inteiro decimal correspondente a representação binária passada no parâmetro `s`. Por exemplo, `fBinToDec ("110")` deve retornar 6.



11. Implementar a função `fEhDataValida (data)` que devolve `true` se a string `data`, no formato `dd/mm/aaaaa`, é uma data válida e `false` em caso contrário. Considere a existência de anos bissextos.
12. Implementar a função `fEhHoraValida (hora)`, que devolve `true` se a string `hora`, no formato `hh:mm:ss`, é uma hora válida e `false` em caso contrário.
13. Implementar a função `fEhUFValida (uf)`, que devolve `true` se a string `uf` é uma sigla válida de uma das unidades federativas brasileiras (PR, SP, RS, SC, RJ, MG, PA, etc.)
14. Implementar a função `fDecToRom (n)`, que devolve a representação romana do decimal inteiro `n`. Por exemplo, `fDecToRom (15)` deve retornar "XV". Os numerais romanos usam 7 símbolos distintos I, V, X, L, C, D e M. Pesquise as regras de formação, incluindo a notação subtrativa.
15. EAN-13 é um código de barras no padrão EAN definido pela GS1 Brasil (Associação Brasileira de Automação), adotado em mais de 100 países membros GS1, para a identificação dos itens, principalmente nos pontos de venda a varejo. No EAN-13 o símbolo codifica treze dígitos que estão divididos em quatro partes; dos treze dígitos, doze são dos dados referentes ao produto e um é o dígito verificador (Codificação EAN-13). As quatro partes que compõem o código são: (1) país de origem do produto, (2) empresa fabricante, (3) produto por ela produzido e (4) dígito verificador. Escreva uma função `fValidaEAN` que recebe uma string representando o código de barras EAN-13 de um produto. A última posição é o dígito verificador (DV). A função deve calcular o DV e retornar um booleano indicando se o código lido está correto ou não. Para calcular o DV considere o seguinte algoritmo:



Tomando como base o código EAN-13 de número: 789100031550-?

- 1) Multiplicam-se os dígitos do código por 1 e por 3, em sequência repetitiva de 1 e 3;

$$7 * 1 = 7$$

$$8 * 3 = 24$$

$$9 * 1 = 9$$



1 * 3 = 3
0 * 1 = 0
0 * 3 = 0
0 * 1 = 0
3 * 3 = 9
1 * 1 = 1
5 * 3 = 15
5 * 1 = 5
0 * 3 = 0

- 2) Soma-se o resultado das multiplicações. Neste caso o resultado da soma é 73.
- 3) Encontra-se o múltiplo de 10 mais próximo do resultado da soma. Atenção: o múltiplo de 10 deve ser maior ou igual ao resultado da soma (nunca menor). Neste caso o múltiplo de 10 maior ou igual e mais próximo de 73 é 80.
- 4) Subtrair o resultado da soma do múltiplo encontrado: $80 - 73 = 7$. O resultado desta subtração é o DV.

Assim o código completo é: **7891000315507**

- 16.** Muitas aplicações utilizam o conceito de *dígito verificador*: um número de uma conta corrente, de um número de matrícula de um estudante de uma escola, o cadastro de pessoas físicas (CPF) mantido pela Receita Federal, etc. O *dígito verificador* serve para a prevenção de possíveis erros de digitação. Por exemplo, se a matrícula 30245-7 fosse digitada erroneamente como 39245-7, o erro seria detectado, pois o *dígito verificador* da conta 39245 seria 6 e não 7. Existem vários métodos para a determinação do *dígito verificador*. Por exemplo, considere a validação do *dígito verificador* de uma conta corrente de uma instituição bancária, conforme o seguinte algoritmo:

1. Multiplicam-se os números correspondentes aos dígitos da conta, da direita para esquerda, por 2, por 3 etc.
2. Somam-se os produtos obtidos no item 1.
3. Determina-se o resto da divisão da soma obtida no item 2 por 11.
4. Subtrai-se de 11 o resto obtido no item 3
5. Se o valor obtido no item 4 for 10 ou 11, o *dígito verificado* é igual a zero; senão, o *dígito* é o valor obtido no item referido.



Por exemplo, se o número da conta for 30245, temos

1. $5 \times 2 = 10$, $4 \times 3 = 12$, $2 \times 4 = 8$, $0 \times 5 = 0$, $3 \times 6 = 18$
2. $10 + 12 + 8 + 0 + 18 = 48$
3. $\text{Resto}(48, 11) = 4$
4. $11 - 4 = 7$
5. Dígito verificador = 7.

Partindo de tais considerações, implemente uma função chamada `fContaValida` que recebe um número de conta e testa o dígito verificador. A função deve retornar um valor lógico, sendo `true` quando o número está correto e `false` caso contrário. Faça um programa para testar a função.

17. Faça uma função que recebe a idade de uma pessoa em anos, meses e dias e retorna essa idade expressa em dias.

18. Faça uma função que verifique se um valor é perfeito ou não. Um valor é dito perfeito quando ele é igual a soma dos seus divisores excetuando ele próprio. (Ex: 6 é perfeito, $6 = 1 + 2 + 3$, que são seus divisores). A função deve retornar um valor booleano.

19. Escreva uma função que recebe dois inteiros positivos e devolve verdadeiro se o produto dos números não se altera invertendo-se os dígitos. Por exemplo, $93 \times 13 = 39 \times 31 = 1\ 209$.

20. As companhias aéreas normalmente representam os nomes dos passageiros no formato último sobrenome/nome. Por exemplo, o passageiro Robert Bruce Banner seria indicado por Banner/Robert. Escreva uma função `fFormataNome(char nome*)` que receba um nome na string `nome` devolve uma string com o nome no formato das companhias aéreas.

21. Escreva uma função que recebe 3 valores reais X, Y e Z e que verifique se esses valores podem ser os comprimentos dos lados de um triângulo e, neste caso, retornar qual o tipo de triângulo formado. Para que X, Y e Z formem um triângulo é necessário que a seguinte propriedade seja satisfeita: o comprimento de cada lado de um triângulo é menor do que a soma do comprimento dos outros dois lados. A função deve identificar o tipo de triângulo formado observando as seguintes definições:



Triângulo Equilátero: os comprimentos dos 3 lados são iguais.

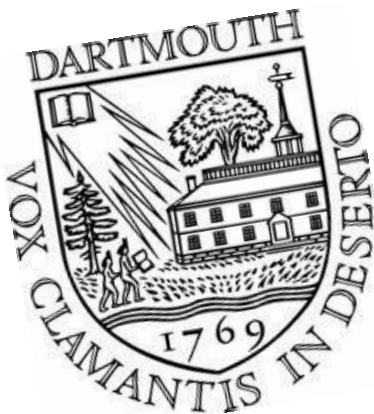
Triângulo Isóscele: os comprimentos de 2 lados são iguais.

Triângulo Escaleno: os comprimentos dos 3 lados são diferentes.

22. Dois números inteiros são ditos amigos se a soma dos divisores de cada um deles (menores que eles) é igual ao outro. Por exemplo, os divisores de 220 são 1, 2, 4, 5, 10, 11, 20, 22, 44, 55 e 110 e $1 + 2 + 4 + 5 + 10 + 11 + 20 + 22 + 44 + 55 + 110 = 284$ e os divisores de 284 são 1, 2, 4, 71 e 142. Somando os divisores temos $1 + 2 + 4 + 71 + 142 = 220$. Escreva uma função chamada `fEhAmigo(n, m)` que testa se n e m são amigos.



23. Escreva uma função que retorne o k -ésimo dígito (da direita para esquerda) de um inteiro n , k e n dados. Por exemplo, $K_esimoDigito(1984, 3) = 9$.
24. Um dos recursos comuns disponibilizados pelos editores de textos modernos é a contagem de palavras de um texto. Escreva uma função que determine o número de palavras de uma frase dada.
25. É muito comum que os títulos de documentos como avisos, declarações, atestados, etc., apareçam em letras maiúsculas separadas por um espaço em branco. Escreva uma função que receba uma palavra e a retorne no formato descrito.
26. Escreva a função $fMaxDivCom(a, b)$ que retorna o máximo divisor comum entre a e b .
27. Em criptografia, a **Cifra de César**, também conhecida como cifra de troca, código de César ou troca de César, é uma das mais simples e conhecidas técnicas de criptografia. É um tipo de cifra de substituição na qual cada letra do texto é substituída por outra, que se apresenta no alfabeto abaixo dela um número fixo de vezes. Por exemplo, com uma troca de três posições, *A* seria substituído por *D*, *B* se tornaria *E*, e assim por diante. O nome do método é em homenagem a **Júlio César** (100 a.C-44 a.C) que o usou para se comunicar com os seus generais. Implemente as funções $fEncryption(msg, shift)$, que devolve uma string contendo o texto plano msg criptografado com a Cifra de César aplicando deslocamento $shift$ e $fDecryption(msg, shift)$ que devolve o texto criptografado msg como texto plano. Teste as funções implementando um script que lê uma frase, apresenta-a encriptada e depois decriptada. Deixe para o usuário escolher o deslocamento desejado, sendo 5 o padrão.



Typing is no substitute for thinking.

Dartmouth BASIC manual, 1964.